

## UNIT II

### Windows Forms

The **Windows Forms** platform supports Windows GUI programs. And the VB.NET language can be used to control these programs. We run arbitrary code as a result of event handlers. This section covers Windows Forms in the VB.NET language.

The whole power of Visual Basic has been that you can develop forms visually, adding controls and other items from the toolbox. In VB .NET, the support for Windows forms is in the **System.Windows.Forms** namespace, and the form class is **System.Windows.Forms.Form**. The **Form** class itself is based on the **Control** class, which means that forms share a lot of the properties and methods that controls do. Here's what the class hierarchy looks like for the **Form** class; every level is derived from the one above it (note that all classes are derived from the **Object** class):

The **Form** class only has one class property, **ActiveForm**, which holds the currently active form for the entire application. If you want to determine what window has the focus (that is, is the target of keystrokes), use the **ActiveForm** property. However, the **Form** class does have many object properties. Some of these object properties are public, some private to the object, and some protected (that is, only accessible to objects of the **Form** class or objects of classes derived from **Form**). When working with forms, one usually uses the public object members (that is, the public properties, methods, and events of **Form** objects); you'll find an overview of the most interesting **Form** public object properties in Table 4.1, and the most interesting **Form** public object methods (recall that methods are the procedures built into a class) in Table 4.2. Note that-as is usual with properties and methods in Visual Basic-not all these properties and methods will be available at the time you're designing your code-some only will be available at run time.

It's worth scanning through these tables to familiarize yourself with what's available-such as the **Icon** property, which sets the icon for the form in Windows, or the **BackColor** property, which sets the background color of the form, and so on.

### Windows forms public object properties.

- 1) **ControlBox** - Gets or sets a value indicating if a control box is displayed
- 2) **IsMdiChild**- Indicates if the form is an MDI child form.
- 3) **IsMdiContainer**- Gets or sets a value indicating if the form is a container for MDI child forms
- 4) **Owner** - Gets or sets the form that owns this form
- 5) **FormBorderStyle**- Gets or sets the border style of the form

### Windows forms public object methods :

- 1) **Activate**- Activates the form (gives it focus and makes it active).
- 2) **AddOwnedForm**- Adds an owned form to this form.
- 3) **Close**- Closes the form.
- 4) **Focus**- Gives the form the focus.
- 5) **Hide**- Hides the form.

### Windows forms public object events :

|                       |                                                              |
|-----------------------|--------------------------------------------------------------|
| <b>Activated</b>      | Occurs when the form is activated in code or by the user.    |
| <b>Click</b>          | Occurs when the form is clicked.                             |
| <b>Closed</b>         | Occurs when the form is closed.                              |
| <b>Closing</b>        | Occurs when the form is closing.                             |
| <b>ControlAdded</b>   | Occurs when a new control is added.                          |
| <b>ControlRemoved</b> | Occurs when a control is removed.                            |
| <b>CursorChanged</b>  | Occurs when the cursor property value has changed.           |
| <b>Deactivate</b>     | Occurs when the form loses focus and is not the active form. |
| <b>DoubleClick</b>    | Occurs when the form is double-clicked.                      |

## Immediate Solutions: Setting Title Bar Text:

Use forms text property to set title for the form during design time. In run time use the following code

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    Text = "Welcome to my Application"  
End Sub
```

**Adding/Removing Min/Max Buttons and Setting a Form's Border:** Forms usually come with minimizing and maximizing buttons, as well as a close box at upper right corner. To remove these buttons, you can set the form's **ControlBox** property to **False**. We can also remove the minimizing and maximizing buttons independently, with the **MaximizeBox** and **MinimizeBox** properties.

## Setting a Form's Border

We can set a form's border style with its **FormBorderStyle** property; here are the possible values for that property:

- **Fixed3D**— A fixed, three-dimensional border.
- **FixedDialog**— A thick, fixed dialog-style border.
- **FixedSingle**— A fixed, single-line border.
- **FixedToolWindow**— A tool window border that is not resizable.
- **None**— No border.
- **Sizable**— A resizable border.
- **SizableToolWindow**— A resizable tool window border.

## Setting Control Tab Order:

The user can move from control to control, highlighting the currently selected control, using the Tab key. We can set the order in which control moves from control to control. Select a control whose tab order you want to set. Next, make sure the control's



**TabStop** property is set to **True**. If this property is **False**, the user cannot reach the control using the Tab key. Now set the control's position in the tab order by setting its **TabIndex** property. The first control in the **Tab** order has a **TabIndex** of 0, the next a **TabIndex** of 1, and so on.

**Setting Forms' Initial Positions:** can use a form's **StartPosition** property to specify its initial position on the screen. The values are

1. **CenterParent**— The form is centered within the bounds of its parent form.
2. **CenterScreen**— The form is centered on the current display and has the dimensions specified in the form's size.
3. **Manual**— The **Location** and **Size** properties of the form will determine its starting position.
4. **WindowsDefaultBounds**— The form is positioned at the Windows default location and has the bounds determined by Windows default.
5. **WindowsDefaultLocation**— The form is positioned at the Windows default location and has the dimensions specified in the form's size.

**Moving and Sizing Forms and Controls in Code:** Use the **SetBounds** method to move forms and controls (and optionally set their dimensions), and the **Size** and **Location** properties to set their dimensions.

set the **Size** property to a new **Size** object, and the **Location** property to a new **Point** object. The dimensions you set in **Size** and **Point** objects are measured in pixels, as are all measurements in Visual Basic; you create these objects by passing x and y dimensions to their class's constructors, like this: **Size(x\_dimension, y\_dimension)** and **Point(x\_location, y\_location)**. (Note that in the Visual Basic screen coordinate system, the upper left of the screen is the origin (0, 0) and that positive x values increase downwards, and positive y values increase to the right.)

Example:

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    Size = New Size(100, 100)
```

```
Location = New Point(0, 0)
Button1.Size = New Size(100, 50)
Button1.Location = New Point(0, 0)
```

End Sub

Use the **SetBounds** method to do the same thing:

Overloads Public Sub SetBounds(ByVal x As Integer, ByVal y As Integer, \_  
ByVal width As Integer, ByVal height As Integer)

Here are the arguments you pass to **SetBounds**:

- **x**— The new **Left** property value of the control.
- **y**— The new **Right** property value of the control.
- **width**— The new **Width** property value of the control.
- **height**— The new **Height** property value of the control.

Ex:

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
ByVal e As System.EventArgs) Handles Button1.Click  
SetBounds(0, 0, 100, 100)  
Button1.SetBounds(0, 0, 100, 50)
```

End Sub

**Showing and Hiding Controls and Forms:** use the control's or form's **Visible** property. Setting this property to **True** displays the control or form; setting it to **False** hides it.

**Ex:**

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
ByVal e As System.EventArgs) Handles Button1.Click  
Button1.Visible = False
```

End Sub

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
ByVal e As System.EventArgs) Handles Button1.Click  
Form2.Hide()  
Form3.Show()
```

End Sub

## Using the *MsgBox* Function:

```
Public Function MsgBox(Prompt As Object [, Buttons As MsgBoxStyle =  
MsgBoxStyle.OKOnly [, Title As Object = Nothing]]) As MsgBoxResultArguments
```

- **Prompt**— A string expression displayed as the message in the dialog box. The maximum length is about 1,024 characters (depending on the width of the characters used).
- **Buttons**— The sum of values specifying the number and type of buttons to display, the icon style to use, the identity of the default button, and the modality of the message box. If you omit *Buttons*, the default value is zero. See below.
- **Title**— String expression displayed in the title bar of the dialog box. Note that if you omit *Title*, the application name is placed in the title bar

### Ex:

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
ByVal e As System.EventArgs) Handles Button1.Click  
Dim Result As Integer  
Result = MsgBox("This is a message box!", MsgBoxStyle.OKCancel +  
MsgBoxStyle.Information + MsgBoxStyle.SystemModal, "Message Box")  
If (Result = MsgBoxResult.OK) Then  
    TextBox1.Text = "You clicked OK"  
End If  
End Sub
```

Result of this code is shown below:





## Using the *MessageBox.Show* Method :

Overloads Public Shared Function Show(ByVal *text* As String, \_  
    ByVal *caption* As String, ByVal *buttons* As MessageBoxButtons, \_  
    ByVal *icon* As MessageBoxIcon, ByVal *defaultButton* As \_  
    MessageBoxDefaultButton, ByVal *options* As MessageBoxOptions \_)  
As DialogResult

### Example:

```
Private Sub Button2_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button2.Click  
    Dim Result As Integer  
    Result = MessageBox.Show("This is a message box!", "Message Box", _  
        MessageBoxButtons.OKCancel, MessageBoxIcon.Information, _  
        MessageBoxDefaultButton.Button1, _  
        MessageBoxOptions.DefaultDesktopOnly)  
    If (Result = DialogResult.OK) Then  
        TextBox1.Text = "You clicked OK"  
    End If  
End Sub
```

Result of this code is shown below:





**Using the *InputBox* Function:** The `InputBox` function is used to get a string of text from the user

### Example

```
Public Function InputBox(Prompt As String [, Title As _  
String = "" [, DefaultResponse As String = "" [, _  
XPos As Integer = -1 [, YPos As Integer = -1]]]) As String
```

- **Prompt**— A string expression displayed as the message in the dialog box. The maximum length is about 1,024 characters (depending on the width of the characters used).
- **Title**— String expression displayed in the title bar of the dialog box. Note that if you omit **Title**, the application name is placed in the title bar.
- **DefaultResponse**— A string expression displayed in the text box as the default response if no other input is provided. Note that if you omit **DefaultResponse**, the displayed text box is empty.
- **XPos**— The distance in pixels of the left edge of the dialog box from the left edge of the screen. Note that if you omit **XPos**, the dialog box is centered horizontally.
- **YPos**— The distance in pixels of the upper edge of the dialog box from the top of the screen. Note that if you omit **YPos**, the dialog box is positioned vertically about one-third of the way down the screen.

**Ex:** Result of this code is shown below:

```
Private Sub Button3_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button3.Click  
    Dim Result As String  
    Result = InputBox("Enter your text!")  
    TextBox1.Text = Result  
End Sub
```



**Working with Multiple Forms:** Simultaneously we work with more than one forms using show and hide methods of the form.



Example:

```
Public Class Form1
    Inherits System.Windows.Forms.Form
    Dim OtherWindow As New Form2()

    Private Sub Form1_Load(ByVal sender As System.Object, _
        ByVal e As System.EventArgs) Handles MyBase.Load
        OtherWindow.Show()

    End Sub

    Private Sub Button1_Click(ByVal sender As System.Object, _
        ByVal e As System.EventArgs) Handles Button1.Click
        TextBox1.Text = OtherWindow.TextBox1.Text
        OtherWindow.Hide()

    End Sub
```

**Using Properties to Communicate between Forms:** We can also use the following code to communicate between forms.

```
Dim OtherWindow As New Form2()

    Private Sub Form1_Load(ByVal sender As System.Object, _
        ByVal e As System.EventArgs) Handles MyBase.Load
        OtherWindow.Show()

    End Sub

    Private Sub Button1_Click(ByVal sender As System.Object, _
        ByVal e As System.EventArgs) Handles Button1.Click
        TextBox1.Text = OtherWindow.TextData
        OtherWindow.Hide()

    End Sub
```

**Setting the Startup Form:** Right-click your project in the Solutions Explorer, select Properties, then select the Common Properties folder and the General item in the box at left. Next, select the Form tha you want to run from the Startup Object drop-down list on the right, click OK.

**Creating Multiple Document Interface (MDI) Applications:** Multiple Document Interface (MDI) forms. MDI frame windows can display multiple child windows inside them; in fact, the Visual Basic IDE itself is an MDI frame window.

To create a MDI Form add a form say, **Form1**, set its **IsMdiContainer** property to **True**.

**Docking and Anchoring Controls:** Docking Gets or sets which control borders are docked to its parent control and determines how a control is resized with its parent.

Anchor refers to the position a control has relative to the edges of the form. A textbox, for example, that is anchored to the left edge of a form will stay in the same position as the form is resized. Docking refers to how much space you want the control to take up on the form. If you dock a control to the left of the form, it will stretch itself to the height of the form, but its width will stay the same. Let's take a look at some examples, to clear things up.

### Creating MDI Child Windows in Code:

```
Public Class Form1
    Inherits System.Windows.Forms.Form
    Dim NumberForms As Integer = 0

    Dim Forms(10) As Form2

    Private Sub MenuItem2_Click(ByVal sender As System.Object, _
        ByVal e As System.EventArgs) Handles MenuItem2.Click
        NumberForms += 1
        Forms(NumberForms) = New Form2()
        Forms(NumberForms).Text = "Document" & Str(NumberForms)
        Forms(NumberForms).MdiParent = Me
        Forms(NumberForms).Show()
    End Sub
End Class
```

### Arranging MDI Child Windows

To arrange MDI child windows, you can use the **LayoutMdi** method:

```
Public Sub LayoutMdi(ByVal value As MdiLayout)
```

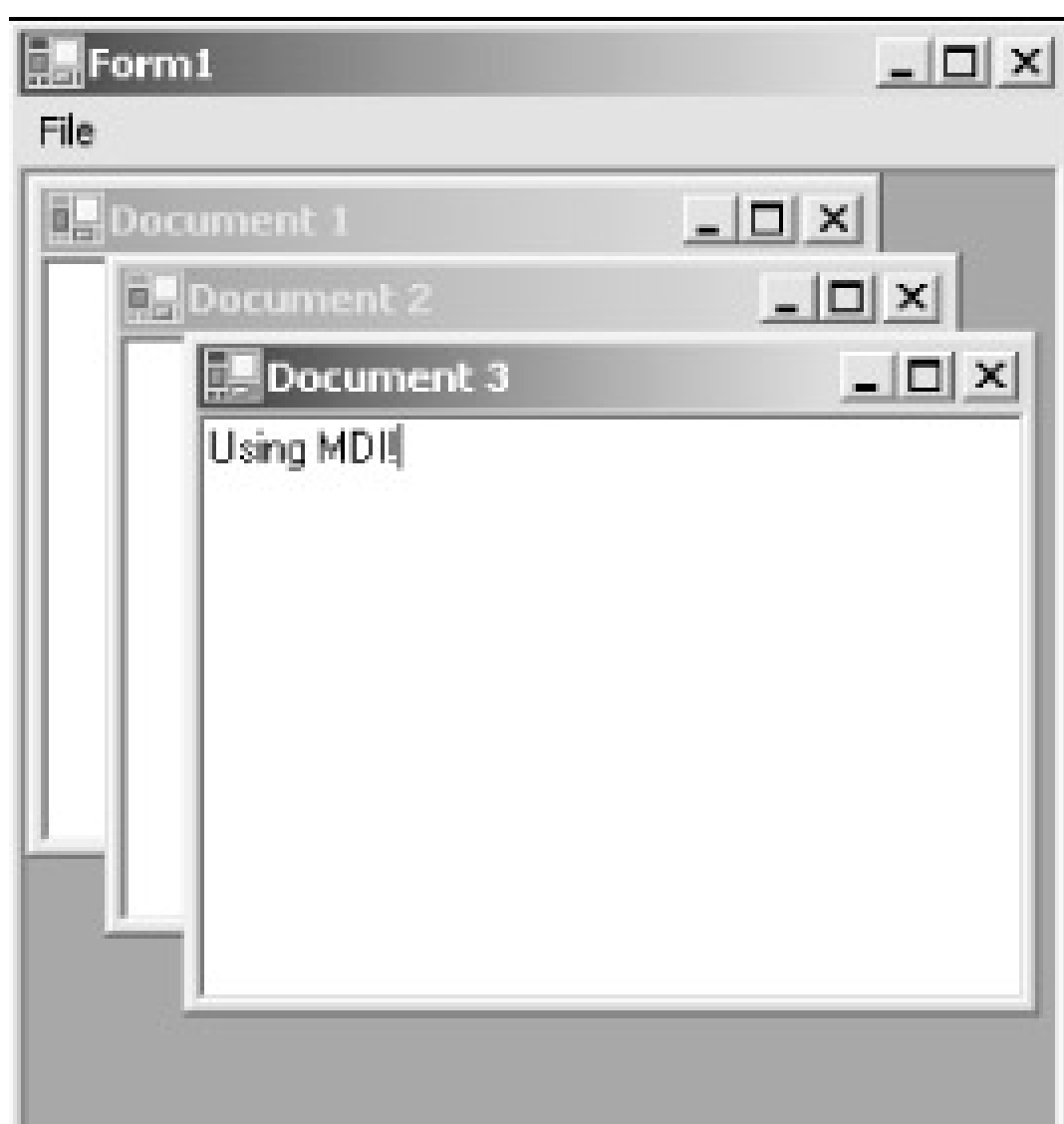
This Sub procedure takes one argument, *value*, which is one of the **MdiLayout** enumeration values that defines the layout of MDI child forms:

- **ArrangeIcons**— All MDI child icons (which are displayed when you minimize an MDI child window) are arranged.
- **Cascade**— All MDI child windows are cascaded.
- **TileHorizontal**— All MDI child windows are tiled horizontally.
- **TileVertical**— All MDI child windows are tiled vertically.

Ex:

```
Private Sub MenuItem3_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles MenuItem3.Click  
    Me.LayoutMdi(MdiLayout.Cascade)  
End Sub
```

**Result is shown below**



**Creating Dialog Boxes:** Dialog boxes are used to interact with the user and retrieve information. In simple terms, a dialog box is a form with its FormBorderStyle Enumeration property set to **FixedDialog**. You can construct your own custom dialog



boxes using the Windows Forms Designer. Add controls such as **Label**, **Textbox**, and **Button** to customize dialog boxes to your specific needs.

To create dialogbox ,add a form set `FormBorderStyle` to **FixedDialog** . Set the **ShowInTaskbar** property of **Form** to **False**. **Add two buttons to the form set text property to Ok and Cancel respectively.** set the **DialogResult** property of the OK button to **OK**, and the same property of the Cancel button to **Cancel**

**Reading Data from Dialog Box:** To read data from dialog box ,first dialogbos must be displayed using **ShowDialog** method, Because **ShowDialog** will return a **DialogResult** value indicating what button the user clicked. wheather the user clicked the OK buttonor cancel button,

```
Public Class Form1
    Inherits System.Windows.Forms.Form
    Dim DialogBox As New Form2()

    'Windows Form Designer generated code

    Private Sub Button1_Click(ByVal sender As System.Object, _
        ByVal e As System.EventArgs) Handles Button1.Click
        If DialogBox.ShowDialog = DialogResult.OK Then
            TextBox1.Text = DialogBox.TextBox1.Text
        End If
    End Sub
End Class
```

**Creating Owned Forms:** An owned form is tied to the form that owns it; if the user minimizes the owner form, the owned form will also be minimized, and so on. You can add an owned form with the **AddOwnedForm** method, and remove an owned form with the **RemoveOwnedForm** method.

**Ex:**

```
Public Class Form1
    Inherits System.Windows.Forms.Form
    Dim OwnedForm1 As New Form2()

    Private Sub Button1_Click(ByVal sender As System.Object, _
        ByVal e As System.EventArgs) Handles Button1.Click
        Me.AddOwnedForm(OwnedForm1)
    End Sub
End Class
```

```
OwnedForm1.Show()  
End Sub  
End Class
```

**Passing Forms to Procedures:** You can pass forms to procedures just as you would any object

**Ex:**

```
Sub ColorWindowRed(ByVal FormToColor As Form)  
    FormToColor.BackColor = System.Drawing.Color.Red  
End Sub
```

**Minimizing/Maximizing and Enabling/Disabling Forms:** We can set the **WindowState** property to maximize or minimize them.

The values of WindowState properties are:

- **FormWindowState.Maximized**- Window is maximized.
- **FormWindowState.Minimized**- Window is minimized.
- **FormWindowState.Normal**- Window is set to normal state

**Ex:**

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    Me.WindowState = FormWindowState.Normal  
End Sub
```

**Adding and Removing Controls at Run Time:** We can add or remove controls to your application at run time, to do is to use the form's **Controls** *collection* 's **Add** or **Remove** methods

**Ex:**

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    Dim NewTextBox As New TextBox()  
    NewTextBox.Size = New Size(100, 20)  
    NewTextBox.Location = New Point(100, 100)
```

```
NewTextBox.Text = "Hello from Visual Basic"  
Me.Controls.Add(NewTextBox)  
End Sub
```

## Handling Mouse Events

You can handle mouse events— such as mouse movements— in forms and controls; here are the possible events for the **Control** class, which is a base class for controls and forms:

- **MouseDown**— Happens when the mouse pointer is over the control and a mouse button is pressed.
- **MouseEnter**— Happens when the mouse pointer enters the control.
- **MouseHover**— Happens when the mouse pointer hovers over the control.
- **MouseLeave**— Happens when the mouse pointer leaves the control.
- **MouseMove**— Happens when the mouse pointer is moved over the control.
- **MouseUp**— Happens when the mouse pointer is over the control and a mouse button is released.
- **MouseWheel**— Happens when the mouse wheel moves while the control has focus.

**Ex:**

```
Private Sub Form1_MouseDown(ByVal sender As Object, _  
    ByVal e As System.Windows.Forms.MouseEventArgs) _  
    Handles MyBase.MouseDown  
    If e.Button = MouseButton.Left Then  
        TextBox1.Text = "Mouse down at " + CStr(e.X) + ", " + CStr(e.Y)  
    End If  
End Sub
```

## Keyboard Events:

- **KeyDown**— Happens when a key is pressed down while the control has focus.
- **KeyPress**— Happens when a key is pressed while the control has focus.
- **KeyUp**— Happens when a key is released while the control has focus

## Sending Keystrokes to Other Programs

Using **SendKeys** function we can send keystrokes to other applications program that currently has the Windows focus, just as if you typed in those applications itself



Example

Public Class Form1

Inherits System.Windows.Forms.Form

'Windows Form Designer generated code

Private Sub Button1\_Click(ByVal sender As System.Object, \_

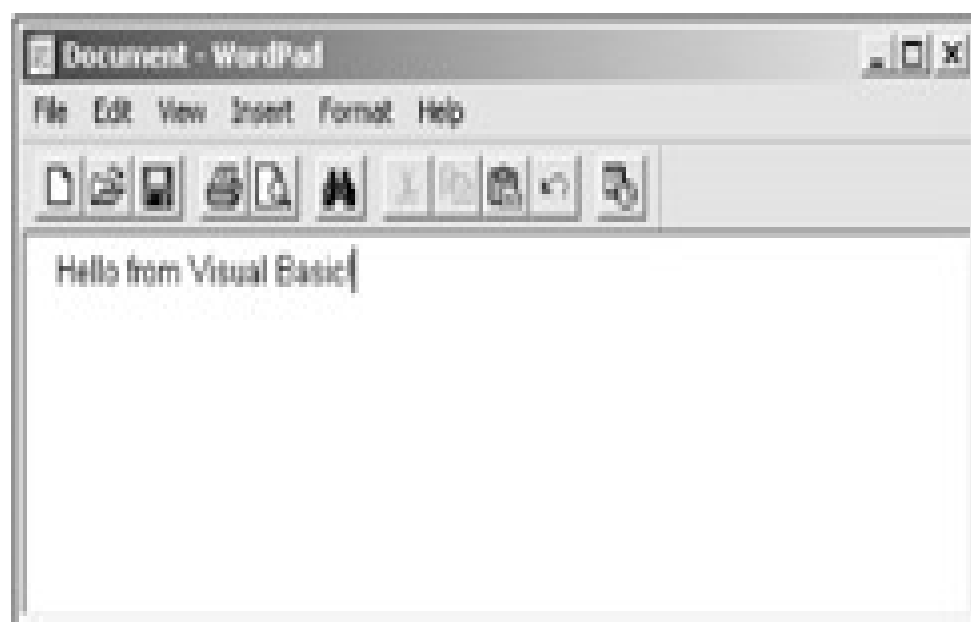
ByVal e As System.EventArgs) Handles Button1.Click

AppActivate("Document - WordPad")

System.Windows.Forms.SendKeys.Send("Hello from Visual Basic!")

End Sub

End Class



**Beeping:** You can use this function to make the computer a beep sound,

Example: When the user clicks a picture box control computer starts beeping

Private Sub PictureBox1\_Click(ByVal sender As System.Object, \_

ByVal e As System.EventArgs) Handles PictureBox1.Click

Beep()

End Sub

## Text Boxes, Rich Text Boxes, Labels, and Link Labels



**The Control Class:** The **Control** class is in the **System.Windows.Forms** namespace. It serves as a base class for the Windows controls

**Setting the Mouse Pointer in Controls:** Using **Cursor** property of any control we can set mouse pointer(pointer shapes)

The some of the values for cursor propetry are listed below

|                |                                                                                      |
|----------------|--------------------------------------------------------------------------------------|
| <b>Arrow</b>   | Arrow cursor.                                                                        |
| <b>Cross</b>   | Crosshair cursor.                                                                    |
| <b>Default</b> | Default cursor, usually an arrow cursor.                                             |
| <b>Hand</b>    | Hand cursor, usually used when hovering over a Web link.                             |
| <b>Help</b>    | Help cursor, a combination of an arrow and a question mark.                          |
| <b>HSplit</b>  | Cursor that appears when the mouse is positioned over a horizontal splitter bar.     |
| <b>IBeam</b>   | I-beam cursor, used to show where the text cursor appears when the mouse is clicked. |

**Text Boxes :** These are box-like controls in which one can enter text

public properties of *TextBox* objects

|                      |                                                                                                                              |
|----------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>AutoSize</b>      | Sets/gets a value specifying if the height of the control automatically adjusts when the font in the control is changed.     |
| <b>BackColor</b>     | Sets/gets the background color of the control.                                                                               |
| <b>BorderStyle</b>   | Sets/gets the border type of the text box control.                                                                           |
| <b>CanUndo</b>       | Returns a value specifying if the user can undo the previous operation.                                                      |
| <b>ForeColor</b>     | Sets/gets the foreground color.                                                                                              |
| <b>HideSelection</b> | Sets/gets a value specifying if the selected text in the text box control remains highlighted when the text box loses focus. |
| <b>Multiline</b>     | Sets/gets a value specifying if this is a multiline text box control.                                                        |
| <b>PasswordChar</b>  | Sets/gets the character used to mask characters of a password in a single-line text box.                                     |
| <b>ReadOnly</b>      | Sets/gets a value specifying if text in the text box is read-only.                                                           |
| <b>ScrollBars</b>    | Sets/gets what scroll bars should appear in a multiline text box.                                                            |

|                        |                                                                          |
|------------------------|--------------------------------------------------------------------------|
| <b>SelectedText</b>    | Sets/gets a value specifying the currently selected text in the control. |
| <b>SelectionLength</b> | Sets/gets the number of characters selected in the text box.             |
| <b>SelectionStart</b>  | Sets/gets the starting point of text selected in the text box.           |
| <b>Text</b>            | Sets/gets the current text in the text box.                              |
| <b>TextAlign</b>       | Sets/gets how text is aligned in a text box control.                     |
| <b>TextLength</b>      | Gets the length of text in the control.                                  |
| <b>WordWrap</b>        | Indicates if a multiline text box control automatically wraps words      |

### public methods of *TextBox* objects

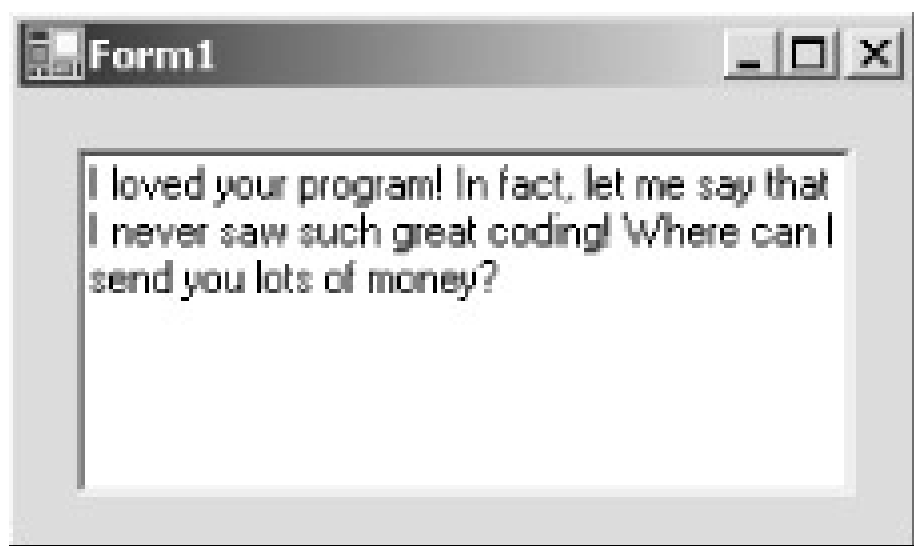
|                      |                                                                                |
|----------------------|--------------------------------------------------------------------------------|
| <b>AppendText</b>    | Appends text to the current text in the text box.                              |
| <b>Clear</b>         | Clears all text from the text box.                                             |
| <b>ClearUndo</b>     | Clears information about the most recent operation of the text box.            |
| <b>Copy</b>          | Copies the selected text in the text box to the Clipboard.                     |
| <b>Cut</b>           | Moves the selected text in the text box to the Clipboard.                      |
| <b>Paste</b>         | Replaces the selected text in the text box with the contents of the Clipboard. |
| <b>ScrollToCaret</b> | Scrolls the text box to the caret position.                                    |
| <b>Select</b>        | Selects text in the text box.                                                  |
| <b>SelectAll</b>     | Selects all text in the text box.                                              |
| <b>Undo</b>          | Undoes the last edit operation in the text box.                                |

### public events of *TextBox* objects

|                        |                                                                   |
|------------------------|-------------------------------------------------------------------|
| <b>AutoSizeChanged</b> | Occurs when the value of the <b>AutoSize</b> property is changed. |
| <b>Click</b>           | Occurs when the text box is clicked.                              |
| <b>ReadOnlyChanged</b> | Occurs when the value of the <b>ReadOnly</b> property is changed. |

### Creating Multiline, Word-wrap Text Boxes:

One can create a textbox that contains text in more than one lines by setting MultiLine property to True as shown below



**Accessing Text in a Text Box:** We can access(read and write) the text contents in textbox using text property.

Example

```
Private Sub Button1_Click_1(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    TextBox1.Text = "Hello from Visual Basic"
End Sub
```

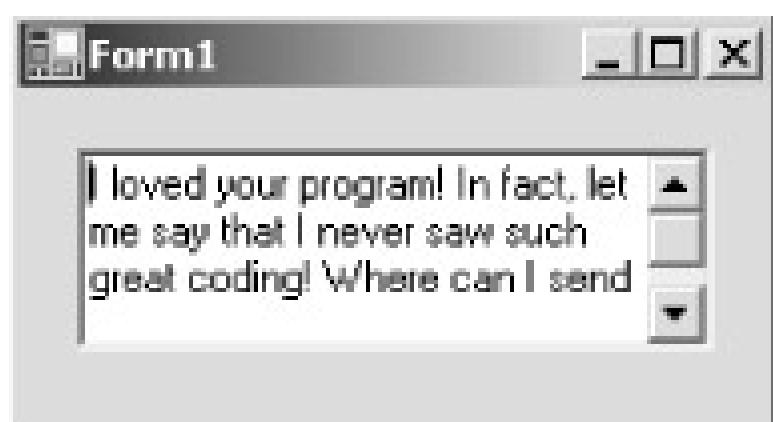
When we execute the above code the text **Hello from Visual Basic** will be placed in textbox

And you can read text from a text box in the same way:

```
Private Sub Button1_Click_1(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    Dim strText As String
    strText = TextBox1.Text
End Sub
```

**Adding Scroll Bars to Text Boxes:** When we are going to enter a lot of text into text boxes, we need not have huge text boxes . using textbox with scrollbars we enter lot of text in such textbox. Using the **ScrollBars** property we can add scrollbar to the textbox

Example : in the following form there is a textbox with vertical scrollbar





The different values of scrollbar property are

- 0: None
- 1: Horizontal
- 2: Vertical
- 3: Both

In order for the scroll bars to actually appear, the text box's **MultiLine** property must be **True**.

**Aligning Text in Text Boxes:** Contents of the textbox can be aligned to centred or leftjustified and so on **TextAlign** property

The values are:

0: left-justified, 1: right-justified, and 2: centered) at design time in all the text boxes

**Making a Text Box Read-only:** Using the *ReadOnly* Property we can make the textbox which does not accept any input from the user. Set **ReadOnly** to **True**

## Disabling a Text Box

You also can disable a text box by setting its **Enabled** property to **False**. However, although this means the user can't enter text into the text box, it also means the text in the box appears grayed. Disabling is better done to indicate that the **control is inaccessible**.

## Using Labels Instead of Text Boxes

Another alternative to using read-only text boxes is to display read-only text in label controls. You can change the text in a label control from code using the label's **Text** property.

## Selecting and Replacing Text in a Text Box

To work with part of the text in a text box, you select the text you want using three properties:

- **SelectionLength**— Returns or sets the number of characters selected.
- **SelectionStart**— Returns or sets the starting point of text selected; indicates the position of the insertion point if no text is selected.

- **SelectedText**— Returns or sets the string containing the currently selected text; consists of a zero-length string ("") if no characters are selected.

**For example,**

here's how we select all the text in a text box and replace it with "Hello from Visual Basic". Note the use of the **Len** function to get the length of the text currently in the text box:

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    TextBox1.SelectionStart = 0
    TextBox1.SelectionLength = Len(TextBox1.Text)
    TextBox1.SelectedText = "Hello from Visual Basic"
End Sub
```

### **The *HideSelection* Property:**

the **HideSelection** property, which, when **True**, turns off text selection highlighting when your program loses the focus

## **Copying or Getting Selected Text to or from the Clipboard**

We can use the **Clipboard** object's **SetDataObject** and **GetDataObject** class methods. The call to **SetDataObject** places the data in the clipboard; **GetDataObject** gets the data from the clipboard; you can check if it is text data with the **GetDataPresent** method and the **DataFormats** enumeration's **Text** item; and you get the actual data with **GetData**:

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    System.Windows.Forms.Clipboard.SetDataObject_
        (TextBox1.SelectedText)
End Sub

Private Sub Button2_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button2.Click
    Dim ClipboardData As System.Windows.Forms.IDataObject = _
        System.Windows.Forms.Clipboard.GetDataObject()
    If ClipboardData.GetDataPresent(DataFormats.Text) Then
        TextBox2.Text = ClipboardData.GetData(DataFormats.Text)
    End If
End Sub
```

## Creating a Password Control

Set **PasswordChar** property to any (specific) character(such as “ \*” ). Whatever the character is typed in to the textbox ,it is converted in to the specified character. Therefore we can not read the contents in the text box as in ordinary textbox .

An example is shown below where textbox contains only \*\*\*\*\*



**Controlling Input in a Text Box:** We can control a textbox in such way that only particular character can be typed in using keypress event of the textbox

Example

to check if the user is typing single digits, you might use this code:

```
Private Sub TextBox1_KeyPress(ByVal sender As Object, _  
    ByVal e As System.Windows.Forms.KeyPressEventArgs) _  
    Handles TextBox1.KeyPress  
    If (e.KeyChar < "0" Or e.KeyChar > "9") Then  
        MsgBox("Please enter single digits")  
    End If  
End Sub
```

If you simply want to enter only digits and nothing else . set the **KeyPressEventArgs Handled** property to **True**

```
Private Sub TextBox1_KeyPress(ByVal sender As Object, _  
    ByVal e As System.Windows.Forms.KeyPressEventArgs) _  
    Handles TextBox1.KeyPress  
    If (e.KeyChar < "0" Or e.KeyChar > "9") Then  
        e.Handled = True  
    End If  
End Sub
```



Creating a Text Box in Code

To create a new text box, use Add() method of **controls collection**

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    Dim TextBox1 As New TextBox()
    TextBox1.Size = New Size(150, 20)
    TextBox1.Location = New Point(80, 20)
    TextBox1.Text = "Hello from Visual Basic"
    Me.Controls.Add(TextBox1)
End Sub
```

Rich Text Boxes

The Windows forms **RichTextBox** control is used for displaying, entering, and manipulating rich (large number of ) text with formatting. The **RichTextBox** control does everything the **TextBox** control does, but in addition, it can display fonts, colors, and links; load text and embedded images from a file; undo and redo editing operations; and find specified characters

The **RichTextBox** control has many properties to format text. You can set font attributes, set indents, create hanging indents, create bulleted paragraphs, and more. To work with files, you can use the **LoadFile** and **SaveFile** methods. You can even use a **RichTextBox** for Web-style links by setting the **DetectUrls** property to **True** and writing code to handle the **LinkClicked** event. And you can undo and redo most edit operations by calling the **Undo** and **Redo** methods; the **CanRedo** method will let you know if the last operation the user has undone can be reapplied.

public properties of *RichTextBox* objects

| Property          | Means                                                                                                      |
|-------------------|------------------------------------------------------------------------------------------------------------|
| AutoSize          | Sets/gets a value specifying if the size of the rich text box automatically adjusts when the font changes. |
| AutoWordSelection | Sets/gets a value specifying if automatic word selection is enabled.                                       |
| BorderStyle       | Sets/gets the border type of the rich text box.                                                            |
| BulletIndent      | Sets/gets the indentation used in the rich text box when the bullet                                        |



| Property             | Means                                                                                                                      |
|----------------------|----------------------------------------------------------------------------------------------------------------------------|
|                      | style is applied to the text.                                                                                              |
| <b>CanRedo</b>       | Indicates if there were actions in rich text box that can be reapplied.                                                    |
| <b>CanUndo</b>       | Returns a value specifying if the user can undo the previous operation in the rich text box.                               |
| <b>DetectUrls</b>    | Sets/gets a value specifying if the rich text box should detect URLs when typed into the <b>RichTextBox</b> control.       |
| <b>HideSelection</b> | Sets/gets a value specifying if the selected text should stay highlighted when the <b>RichTextBox</b> control loses focus. |
| <b>Lines</b>         | Sets/gets the lines of text in a <b>RichTextBox</b> control.                                                               |
| <b>MaxLength</b>     | Sets/gets the maximum number of characters the user can type into the rich text box.                                       |

public methods of *RichTextBox* objects.

| Method            | Means                                                                                         |
|-------------------|-----------------------------------------------------------------------------------------------|
| <b>AppendText</b> | Appends text to the current text of the rich text box.                                        |
| <b>CanPaste</b>   | Determines if you can paste information from the Clipboard.                                   |
| <b>Clear</b>      | Clears all text from the <b>RichTextBox</b> control.                                          |
| <b>ClearUndo</b>  | Clears information about the most recent operation from the undo buffer of the rich text box. |
| <b>Copy</b>       | Copies the current selection in the rich text box to the Clipboard.                           |
| <b>Cut</b>        | Moves the current selection in the rich text box to the Clipboard.                            |
| <b>Find</b>       | Searches for text within the contents of the rich text box.                                   |
| <b>LoadFile</b>   | Loads the contents of a file into the RichTextBox control.                                    |

| Method           | Means                                                                   |
|------------------|-------------------------------------------------------------------------|
| <b>Paste</b>     | Pastes the contents of the Clipboard into the RichTextBox control.      |
| <b>Redo</b>      | Reapplies the last operation that was undone in the RichTextBox control |
| <b>SaveFile</b>  | Saves the contents of the rich text box to a file                       |
| <b>Select</b>    | Selects text within the RichTextBox control.                            |
| <b>SelectAll</b> | Selects all text in the rich text box.                                  |
| <b>Undo</b>      | Undoes the last edit operation in the rich text box                     |

public events of *RichTextBox* objects.

|                         |                                                                                          |
|-------------------------|------------------------------------------------------------------------------------------|
| <b>Click</b>            | Occurs when the rich text box is clicked.                                                |
| <b>LinkClicked</b>      | Occurs when the user clicks on a link within the text of the <b>RichTextBox</b> control. |
| <b>ModifiedChanged</b>  | Occurs when the value of the <b>Modified</b> property is changed.                        |
| <b>ReadOnlyChanged</b>  | Occurs when the value of the <b>ReadOnly</b> property is changed.                        |
| <b>SelectionChanged</b> | Occurs when the selection of text within the <b>RichTextBox</b> control is changed.      |
| <b>VScroll</b>          | Occurs when the user clicks the vertical scroll bars of the <b>RichTextBox</b> control   |

### Accessing Text in a Rich Text Box

To access text in a rich text box, you can use two properties: **Text** and **Rtf**. As their names imply, **Text** holds the text in a rich text box in plain text format (like a text box), and **Rtf** holds the text in rich text format.

Here's an example where we read the text in **RichTextBox1** without any RTF codes and display that text as plain text in **RichTextBox2**:

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    RichTextBox2.Text = RichTextBox1.Text
End Sub
```

Here's the same operation where we transfer the text including all RTF codes-that is, here we're transferring rich text from one rich text box to another:

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button2.Click
    RichTextBox2.Rtf = RichTextBox1.Rtf
End Sub
```

**Creating Bold, Italic, Underlined, and Strikeout Text:** First, we select the text you want to format, using **Find** method, setting the return value of that method to the **SelectionStart** property. That selects the text to format.

## Example

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    RichTextBox1.SelectionStart = RichTextBox1.Find("italic")
    Dim ItalicFont As New Font(RichTextBox1.Font, FontStyle.Italic)
    RichTextBox1.SelectionFont = ItalicFont
```

```
RichTextBox1.SelectionStart = RichTextBox1.Find("bold")
Dim BoldFont As New Font(RichTextBox1.Font, FontStyle.Bold)
RichTextBox1.SelectionFont = BoldFont
```

```
RichTextBox1.SelectionStart = RichTextBox1.Find("underlined")
Dim UnderlineFont As New Font(RichTextBox1.Font, FontStyle.Underline)
RichTextBox1.SelectionFont = UnderlineFont
```

```
RichTextBox1.SelectionStart = RichTextBox1.Find("strikeout")
Dim StrikeoutFont As New Font(RichTextBox1.Font, FontStyle.Strikeout)
RichTextBox1.SelectionFont = StrikeoutFont
```

```
End Sub
```

**Indenting Text in Rich Text Boxes:** use the following properties to set indent text in richtextbox

1. **SelectionIndent—** Indents first line of the paragraph
2. **SelectionHangingIndent—** Indents all other lines of the paragraph with respect to **SelectionIndent**
3. **SelectionRightIndent—** Sets the right indentation of the paragraph

## Example



```
Private Sub Button3_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button3.Click
    RichTextBox1.SelectionIndent = 20
    RichTextBox1.SelectionHangingIndent = - 25
    RichTextBox1.SelectionRightIndent = 10
End Sub
```

## Adding Bullets to Rich Text Boxes

Rich text boxes support *bullets*— the lists of items that you want to display . Putting a bullet in front of each item gives the list a snappy appearance and can be very effective visually.

To set bullets, you use the **SelectionBullet** and **BulletIndent** properties. The **SelectionBullet** property displays a bullet in front of the paragraph in which the current selection is, and the **BulletIndent** property indicates how much you want the bullet to be indented from the left.

### Here's an example from the RichTextBoxes

```
Private Sub Button4_Click(ByVal sender As System.Object, ByVal e As _
    System.EventArgs) Handles Button4.Click
    RichTextBox1.Text = "This rich text box shows how to use bullets " + _
        "and indent bulleted text."
    RichTextBox1.SelectionIndent = 20
    RichTextBox1.BulletIndent = 10
    RichTextBox1.SelectionBullet = True
End Sub
```

### Setting Text Color in RTF Boxes

Using the **SelectionColor** property, we can set colors in a rich text box,

#### Example

```
Private Sub Button5_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button5.Click
    RichTextBox3.SelectionStart = RichTextBox3.Find("red")
    RichTextBox3.SelectionColor = Color.Red
End Sub
```

```
RichTextBox3.SelectionStart = RichTextBox3.Find("green")
RichTextBox3.SelectionColor = Color.Green
```

```
RichTextBox3.SelectionStart = RichTextBox3.Find("blue")
RichTextBox3.SelectionColor = Color.Blue
```

End Sub

## Saving and Loading RTF Files from and to Rich Text Boxes

**SaveFile** method enables us to save the text in a rich text box to disk, and the **LoadFile** method to read it back.

Example

```
Private Sub Button6_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button6.Click
    RichTextBox3.SaveFile("text.rtf")
    RichTextBox1.LoadFile("text.rtf")
End Sub
```

## Aligning Text in a Rich Text Box

You can set the alignment of text in a rich text box paragraph by paragraph using the **SelectionAlignment** property. You just select the paragraph you want to align— or place the insertion point in that paragraph— and set the **SelectionAlignment** property to one of the following values:

- **HorizontalAlignment.Left**— 0 (the default)— The paragraph is aligned along the left margin.
- **HorizontalAlignment.Right**— 1— The paragraph is aligned along the right margin.
- **HorizontalAlignment.Center**— 2— The paragraph is centered between the left and right margins.

## Creating Rich Text Boxes in Code

Creating rich text boxes in code works much like creating standard text boxes in

Example : creates a new rich text box when the user clicks a button:

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    Dim RichTextBox1 As New RichTextBox()
    RichTextBox1.Size = New Size(150, 100)
```

```

RichTextBox1.Location = New Point(70, 20)
RichTextBox1.Text = "Hello from Visual Basic"
Me.Controls.Add(RichTextBox1)
End Sub

```

## Labels:

Labels usually are used to display text that cannot be edited by the user. Your code can change the text displayed by a label. Labels are based directly on the **Control** class

### public properties of *Label* objects

|                        |                                                                                                          |
|------------------------|----------------------------------------------------------------------------------------------------------|
| <b>AutoSize</b>        | Sets/gets a value specifying if the control should be automatically resized to display all its contents. |
| <b>BorderStyle</b>     | Sets/gets the border style for the control.                                                              |
| <b>FlatStyle</b>       | Sets/gets the flat style appearance of the label control.                                                |
| <b>Image</b>           | Sets/gets the image that is displayed on a <b>Label</b> .                                                |
| <b>ImageAlign</b>      | Sets/gets the alignment of an image that is displayed in the control.                                    |
| <b>PreferredHeight</b> | Gets the preferred height of the control.                                                                |
| <b>PreferredWidth</b>  | Gets the preferred width of the control.                                                                 |
| <b>TextAlign</b>       | Sets/gets the alignment of text in the control.                                                          |

## Using Labels Instead of Text Boxes

Labels display read-only text and they give the appearance of text directly on the form; this can look much better than a text box on occasion. For example, you might want to display the result of a calculation in a label instead of a text box, so the user can't edit that result. To make a label look like a text box, set its **BackColor** property to **Window** and its **BorderStyle** property to **Fixed3D**.

**Formatting Text in Labels:** When you add labels to a form, you can make the label match the text's size by setting the **AutoSize** property to **True**. You also can format the text in a label with the **Font** property, setting it to a **Font** object



## Aligning Text in Labels

As with text boxes, you can align text in labels. set the label's **TextAlign** property at design time or run time. This property takes values from the **ContentAlignment** enumeration:

- **BottomCenter**— Vertically aligned at the bottom, and horizontally aligned at the center.
- **BottomLeft**— Vertically aligned at the bottom, and horizontally aligned on the left.
- **BottomRight**— Vertically aligned at the bottom, and horizontally aligned on the right.
- **MiddleCenter**— Vertically aligned in the middle, and horizontally aligned at the center.
- **MiddleLeft**— Vertically aligned in the middle, and horizontally aligned on the left.
- **MiddleRight**— Vertically aligned in the middle, and horizontally aligned on the right.
- **TopCenter**— Vertically aligned at the top, and horizontally aligned at the center.
- **TopLeft**— Vertically aligned at the top, and horizontally aligned on the left.
- **TopRight**— Vertically aligned at the top, and horizontally aligned on the right.

**Handling Label Events:**label have events like Click and DoubleClick **Here's an example using the DoubleClick event to change the text in the label when the user double-clicks it:**

```
Private Sub Label1_DoubleClick(ByVal sender As Object,
    ByVal e As System.EventArgs) Handles Label1.DoubleClick
    Label1.Text = "Quit clicking me!"
End Sub
```

**Using Labels to Give Access Keys to Controls without Captions:** By placing an ampersand (&) in the caption in front of the letter you want to make the access key. The access key appears underlined and the corresponding control is given the focus if the user presses Alt and the access key. you give the access key to a label control, and then make sure the control you want to give the focus to with that access key is next in the tab order (i.e., has the next highest TabIndex property value). Because labels cannot accept the focus themselves,when the user presses Alt and the access key, the label passes the focus on to the next control

**Link Labels:** Link Labels support Web-style hyperlinks to the Internet and other Windows forms

**public properties of *LinkLabel* objects:**

|                        |                                         |
|------------------------|-----------------------------------------|
| <b>ActiveLinkColor</b> | Sets/gets the color for an active link. |
|------------------------|-----------------------------------------|



|                          |                                                                                           |
|--------------------------|-------------------------------------------------------------------------------------------|
| <b>r</b>                 |                                                                                           |
| <b>DisabledLinkColor</b> | Sets/gets the color for a disabled link.                                                  |
| <b>LinkArea</b>          | Sets/gets the range in the text to treat as a link.                                       |
| <b>LinkBehavior</b>      | Sets/gets a value that represents the behavior of a link.                                 |
| <b>LinkColor</b>         | Sets/gets the color for a normal link.                                                    |
| <b>Links</b>             | Gets the collection of links in the <b>LinkLabel</b> control.                             |
| <b>LinkVisited</b>       | Sets/gets a value specifying if a link should be displayed as though it had been visited. |
| <b>VisitedLinkColor</b>  | Sets/gets the color used for links that that have been visited.                           |

public events of *LinkLabel* objects.

**LinkClicked:-** Occurs when a link is clicked inside the link label

**Creating a LinkLabel :** **LinkArea** property in the properties window and clicking the ellipsis ("... ") button that appears to open the LinkArea editor. Select the area you want to make into a hyperlink there, and click OK. You can also set the **LinkColor**, **VisitedLinkColor**, and **ActiveLinkColor** properties to set the colors of the hyperlink.

Ex:

```
Private Sub LinkLabel1_LinkClicked(ByVal sender As System.Object, _
    ByVal e As System.Windows.Forms.LinkLabelLinkClickedEventArgs) _
    Handles LinkLabel1.LinkClicked
    LinkLabel1.LinkVisited = True
    System.Diagnostics.Process.Start("www.coriolis.com")
End Sub
```

**Creating a *LinkLabel* in Code:** Execute the following code to create a link label during run time

```
Creating a LinkLabel in Code: LinkLabel1 = New LinkLabel()
LinkLabel1.AutoSize = True
LinkLabel1.Location = New Point(15, 15)
LinkLabel1.Size = New Size(135, 15)
LinkLabel1.Text = "Interested in Black Books? Click here to see them all!"
```

```
LinkLabel1.Links.Add(14, 11, "info")
LinkLabel1.Links.Add(33, 4, "www.coriolis.com")
```

## Linking to Another Form:

get the text associated with the link with the `e.Link.LinkData.ToString` method and use this method to link to desired form

```
Private Sub LinkLabel1_LinkClicked(ByVal sender As Object, _
    ByVal e As System.Windows.Forms.LinkLabelLinkClickedEventArgs)
    LinkLabel1.Links(LinkLabel1.Links.IndexOf(e.Link)).Visited = True
    If (e.Link.LinkData.ToString() = "info") Then
        Dim InfoWindow As New Form2()
        InfoWindow.Show()
    End If
End Sub
```

## Linking to the Web:

Use the following code to open any web page or web site

```
System.Diagnostics.Process.Start(" URL" )
```

Example:

```
Private Sub LinkLabel1_LinkClicked(ByVal sender As Object, _
    ByVal e As System.Windows.Forms.LinkLabelLinkClickedEventArgs)
    LinkLabel1.Links(LinkLabel1.Links.IndexOf(e.Link)).Visited = True
    If (e.Link.LinkData.ToString() = "info") Then
        Dim InfoWindow As New Form2()
        InfoWindow.Show()
    Else
        System.Diagnostics.Process.Start(e.Link.LinkData.ToString())
    End If
End Sub
```

# Windows Forms: Buttons, Checkboxes, Radio Buttons, Panels, and Group Boxes

## Buttons

Buttons are the plain controls that you simply click and release. Buttons can be clicked with the mouse or with the Enter key if the button has the focus

## Checkboxes

Checkboxes are also familiar controls— you click a checkbox to select it, and click it again to deselect it. When you select a checkbox, a check appears in it, indicating that the box is indeed selected

## Radio buttons

Radio buttons, also called option buttons, are similar to checkboxes— the user can select and deselect them

## panels

You use panels to group other controls, usually to divide a form into regions by function

## Group Boxes

Like panels, group boxes are used to provide a grouping for other controls. Group boxes are similar to panels, but, as mentioned above, only group boxes display captions and only the panels can have scroll bars

## All About Buttons:

Like other Windows controls, the **Button** class is based on the **Control** class

Noteworthy public properties of *Button* objects

|                     |                                                                                                                           |
|---------------------|---------------------------------------------------------------------------------------------------------------------------|
| <b>DialogResult</b> | Gets/sets the value returned to the parent form when the button is clicked. Often used when you're creating dialog boxes. |
|---------------------|---------------------------------------------------------------------------------------------------------------------------|



|                   |                                                                                |
|-------------------|--------------------------------------------------------------------------------|
| <b>FlatStyle</b>  | Gets/sets a flat style appearance.                                             |
| <b>Image</b>      | Gets/sets an image displayed in a button.                                      |
| <b>ImageAlign</b> | Gets/sets the alignment of the image in a button.                              |
| <b>ImageIndex</b> | Gets/sets the image list index value of the image displayed in the button.     |
| <b>ImageList</b>  | Gets/sets the <b>ImageList</b> that contains the images displayed in a button. |
| <b>TextAlign</b>  | Gets/sets the alignment of the text in the button.                             |

Noteworthy public methods of *Button* objects

**PerformClick** :- Causes a **Click** event for a button.

## Setting a Button's Caption

use a button's **Text** to set its caption. This property is available at both design time and run time. After you add a button to a form, you set its caption by placing the appropriate text in the **Text** property in the properties window:

or in code use the following code

```
Button1.Text = "text to be displayed"
```

example

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    Button1.Text = "You clicked me!"  
End Sub
```

## Setting a Button's Foreground and Background Color

You use the button's **Background** and **ForeColor** property to set Foreground and Background Color respectively

Example:

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    Button1.BackColor = Color.Blue  
    Button1.ForeColor = Color.Red
```

End Sub

**Setting Button Fonts:** you use the button's **Font** property to set font . Also we set font in code as follows

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    Button1.Font = New Font(Button1.Font, FontStyle.Italic)
```

End Sub

## Handling Button Clicks

To add a click event handler, just double-click the button at design time

Example

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click
```

End Sub

Place the code you want to execute when the button is clicked in this Sub procedure as shown below

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    TextBox1.Text = "Hello from Visual Basic"
```

End Sub

Here, the **sender** argument is the button object itself that caused the event, and the **e** argument is a simple **EventArgs** object

**Imitating Control Arrays:** Control arrays don't exist in Visual Basic .NET. we create control array in code and give them all the same event handler,. Here is an example where control array for buttons is created using event same handler **Button\_Click**.

Because the actual button that caused the event is passed to the event handler, thereby we can determine which button was clicked

```
Private Sub Button4_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button4.Click  
    Button1 = New Button()  
    Button2 = New Button()  
    Button3 = New Button()  
    :  
    AddHandler Button1.Click, AddressOf Button_Click  
    AddHandler Button2.Click, AddressOf Button_Click  
    AddHandler Button3.Click, AddressOf Button_Click  
End Sub
```

```
Private Sub Button_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs)  
    If sender Is Button1 Then  
        TextBox1.Text = "You clicked button 1"  
    End If  
    If sender Is Button2 Then  
        TextBox1.Text = "You clicked button 2"  
    End If  
    If sender Is Button3 Then  
        TextBox1.Text = "You clicked button 3"  
    End If  
End Sub
```

**Resetting the Focus after a Button Click:** When you click a button, the input focus is transferred to the button but in some cases, you want to set focus to other control. In such a situation use focus method of the control for which you want to set the focus

Example

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    RichTextBox1.Find("Visual Basic")  
    RichTextBox1.Focus() ' focus is sent to the richtext box
```



End Sub

## Giving Buttons and Other Controls Access Characters

place an ampersand (&) in front of the character in the text property text. For example, placing & in front of the word Me in Click Me (i.e., Click &Me), makes M the access character for that button. (Be sure that the access character you choose is unique among all the access characters available at one time.) Under line appears before M and we use Alt and M key to access the button

## Setting Button Tab Order

To make your buttons more accessible from the keyboard when we have lot of buttons in a form , use the **TabStop** and **TabIndex** properties.

- **TabStop** indicates if this button can accept the focus when the user tabs to it.
- **TabIndex** is the index of the current button in the tab order (starts at 0).

## Disabling Buttons

We can disable the button by setting its **Enabled** property to **False** when it's inappropriate to use that button. When a button is disabled, it is inaccessible to the user (and it can't accept the focus).

Example

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    Button1.Enabled = False  
End Sub
```

## Showing and Hiding Buttons

To make a button disappear, just set its **Visible** property to **False**. To make it reappear, set the **Visible** property to **True**. You can set this property at either design time or run time. Here is an example to make a button disappear when you it is clicked

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    Button1.Visible = False  
End Sub
```



## Resizing and Moving Buttons from Code

In VB .NET, you use the **SetBounds** method to move forms and controls (and optionally set their dimensions), and the **Size** and **Location** properties to set their dimensions.

You set the **Size** property to a new **Size** object, and the **Location** property to a new **Point** object. The dimensions you set in **Size** and **Point** objects are measured in pixels, as are all measurements in Visual Basic, and you create these objects by passing x and y dimensions to their class's constructors like this: **Size(x\_dimension, y\_dimension)** and **Point(x\_location, y\_location)**. Note that in the Visual Basic screen coordinate system, the upper left of the screen is the origin (0, 0) and that positive x values increase downward, and positive y values increase to the right.

Here's an example: Say that you wanted to change the size and location of a button in the form when the user clicks that button.

```
Private Sub Button1_Click(ByVal sender As System.Object, ByVal e As
System.EventArgs) Handles Button1.Click
    Button1.Size = New Size(100, 50)
    Button1.Location = New Point(0, 0)
End Sub
```

You also can use the **SetBounds** method to do the same thing; this method is overloaded and has several forms— here's a popular one:

```
Overloads Public Sub SetBounds(ByVal x As Integer, ByVal y As Integer, _
    ByVal width As Integer, ByVal height As Integer)
```

Where,

- **x**— The new **Left** property value of the control.
- **y**— The new **Top** property value of the control.
- **width**— The new **Width** property value of the control.
- **height**— The new **Height** property value of the control.

## Adding a Picture to a Button

We can display images in button. You can add images to buttons at design time and at run time. At design time, you need to set the **Image** property in the Properties window to an image file. At run time, you can do the same thing if you use the **Image** class's **FromFile** class method and assign the resulting **Image** object to the button's **Image** property

Example

```
Button1.Image = _  
    Image.FromFile("C:\vbnet\ch06\imagebuttons\clicked.jpg")
```

## Adding Buttons at Run Time

First declare a variable of type button as follows

```
Button1 = New Button()  
    Button2 = New Button()  
    Button3 = New Button()
```

Specify size and location for the buttons as follows

```
Button1.Size = New Size(80, 30)  
Button1.Location = New Point(115, 20)  
Button1.Text = "Button 1 "
```

```
Button2.Size = New Size(80, 30)  
Button2.Location = New Point(115, 60)  
Button2.Text = "Button 2 "
```

```
Button3.Size = New Size(80, 30)  
Button3.Location = New Point(115, 100)  
Button3.Text = "Button 3 "
```

Use add methods of control class as follows

```
Controls.Add(Button1)  
Controls.Add(Button2)  
Controls.Add(Button3)
```

**Passing Buttons to Procedures:** We can pass buttons to a procedure just like a variable. Here is an example where **button1** is passed to a procedure called **SetCaption** to set text property

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    SetCaption(Button1) ' call to procedure
End Sub
```

```
Private Sub SetCaption(ByRef TargetButton As Button) ' called procedure
    TargetButton.Text = "I've been clicked"
End Sub
```

## Handling Button Releases

Like **Click** event, we can handle mouse release events such as **MouseUp** event. In fact, buttons support the **MouseDown**, **MouseMove**, **MouseUp**, **KeyDown**, **KeyPress**, and **KeyUp** events.

To determine when a button has been released, you can just use its **MouseUp** as follows

```
Private Sub Button1_MouseUp(ByVal sender As Object, _
    ByVal e As System.Windows.Forms.MouseEventArgs) _
    Handles Button1.MouseUp
    TextBox1.Text = "The button went up."
End Sub
```

**Using the *Checkbox* Class:** Checkboxes are based on the **CheckBox** class. The public properties of *Checkbox* objects are given below

|                   |                                                                                                                                                   |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Appearance</b> | Gets/sets the appearance of a checkbox.                                                                                                           |
| <b>AutoCheck</b>  | Specifies if the <b>Checked</b> or <b>CheckState</b> values and the checkbox's appearance are automatically changed when the checkbox is clicked. |
| <b>CheckAlign</b> | Gets/sets the horizontal and vertical alignment of a checkbox in a checkbox control.                                                              |



|                       |                                                                                 |
|-----------------------|---------------------------------------------------------------------------------|
| <b>Checked</b>        | Gets/sets a value indicating if the checkbox is in the checked state.           |
| <b>CheckState</b>     | Gets/sets the state of a three-state checkbox.                                  |
| <b>FlatStyle</b>      | Gets/sets the flat style appearance of the checkbox.                            |
| <b>Image</b>          | Gets/sets the image that is displayed in a checkbox.                            |
| <b>ImageAlign</b>     | Gets/sets the alignment of the image on the checkbox.                           |
| <b>ImageListIndex</b> | Gets/sets the image list index value of the image displayed in the checkbox.    |
| <b>ImageList</b>      | Gets/sets the <b>ImageList</b> that contains the image displayed in a checkbox. |

**public events of *Checkbox* objects:**

|                          |                                                     |
|--------------------------|-----------------------------------------------------|
| <b>AppearanceChanged</b> | Occurs when the <b>Appearance</b> property changes. |
| <b>CheckedChanged</b>    | Occurs when the <b>Checked</b> property changes.    |
| <b>CheckStateChanged</b> | Occurs when the <b>CheckState</b> property changes. |

|  |  |  |
|--|--|--|
|  |  |  |
|--|--|--|

**Creating Checkboxes**

You can handle checkbox **CheckChanged** events, which happen when the **Checked** property changes.

Example

```
Private Sub CheckBox1_CheckedChanged(ByVal sender As System.Object, _
```

```
    ByVal e As System.EventArgs) Handles CheckBox1.CheckedChanged  
    TextBox1.Text = "You clicked check box 1"
```

End Sub

## Getting a Checkbox's State

You can see if a checkbox is checked by examining its **Checked** property. This property can be set to either **True** or **False**

Example will change a button's caption if a checkbox, **CheckBox1**, is checked, but not otherwise:

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    If CheckBox1.Checked Then  
        Button1.Text = "The check mark is checked"  
    End If  
End Sub
```

## Setting a Checkbox's State

You can set a checkbox's state by setting its **Checked** property to **True** or **False**, as in the following code:

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    CheckBox1.Checked = True  
End Sub
```

If the the above code is executed a check mark appears in the checkbox

## Creating Three-State Checkboxes

By default, checkboxes are two-state controls; you use the **Checked** property to get or set the value of a two-state checkbox. However, if you set the checkbox's **ThreeState** property to **True**, you make the checkbox into a three-state control.

You use the **CheckState** property to get or set the value of the three-state checkbox. The three states are:

- **Checked**— A check appears in the checkbox.
- **Unchecked**— No check appears in the checkbox.
- **Indeterminate**— A check appears in the checkbox on a gray background.

If set the checkbox's **ThreeState** property to **True**, you can set its **CheckState** property to **CheckState.Indeterminate** at design time or run time to set the checkbox to the indeterminate state as follows

```
Private Sub CheckBox1_CheckedChanged(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles CheckBox1.CheckedChanged  
    TextBox1.Text = "You clicked checkbox 1 "  
    CheckBox1.CheckState = CheckState.Indeterminate  
End Sub
```

## Using the *RadioButton* Class

Radio buttons are much like checkboxes, except that they're usually used in groups

**public properties of *RadioButton* objects.**

|                   |                                                                                                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Appearance</b> | Gets/sets the value that determines the appearance of the radio button.                                                                                  |
| <b>AutoCheck</b>  | Gets/sets a value indicating whether the <b>Checked</b> value and the appearance of the   control automatically change when the radio button is clicked. |
| <b>Checked</b>    | Gets/sets a value indicating whether the radio button is checked.                                                                                        |
| <b>FlatStyle</b>  | Gets/sets the flat style appearance of the radio button.                                                                                                 |
| <b>Image</b>      | Gets/sets the image that is displayed in a radio button.                                                                                                 |
| <b>ImageAlign</b> | Gets/sets the alignment of the image in a radio button.                                                                                                  |
| <b>ImageIndex</b> | Gets/sets the image list index value of the image displayed in a radio button.                                                                           |
| <b>ImageList</b>  | Gets/sets the <b>ImageList</b> that contains the image displayed in a radio button.                                                                      |



|           |                                                        |
|-----------|--------------------------------------------------------|
| List      |                                                        |
| TextAlign | Gets/sets the alignment of the text in a radio button. |

public methods of *RadioButton* objects.

|              |                                                                                    |
|--------------|------------------------------------------------------------------------------------|
| PerformClick | Generates a <b>Click</b> event for the radio button, simulating a click by a user. |
|--------------|------------------------------------------------------------------------------------|

public events of *RadioButton* objects.

|                   |                                                               |
|-------------------|---------------------------------------------------------------|
| AppearanceChanged | Occurs when the <b>Appearance</b> property changes.           |
| CheckedChanged    | Occurs when the value of the <b>Checked</b> property changes. |
|                   |                                                               |

### Creating Radio Buttons

You can handle radio button **CheckChanged** events, which happen when the **Checked** property changes; here's some code from the RadioButtons example on the CD-ROM:

```
Private Sub RadioButton1_CheckedChanged(ByVal sender As
System.Object, _
    ByVal e As System.EventArgs) Handles
RadioButton1.CheckedChanged
    TextBox1.Text = "You clicked radio button 1"
End Sub
```

### Setting a Radio Button's State

Using the **Checked** property The **Checked** property can take two values, **True** or **False** . As an example set a radio button, **RadioButton1**, to its selected state by setting its **Checked** property to **True**:

```
Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    RadioButton1.Checked = True
End Sub
```



## Getting a Radio Button's State:

You can check if a radio button is selected or not with the **Checked**. Radio buttons' **Checked** property only has two settings: **True** if the button is selected, and **False** if not.

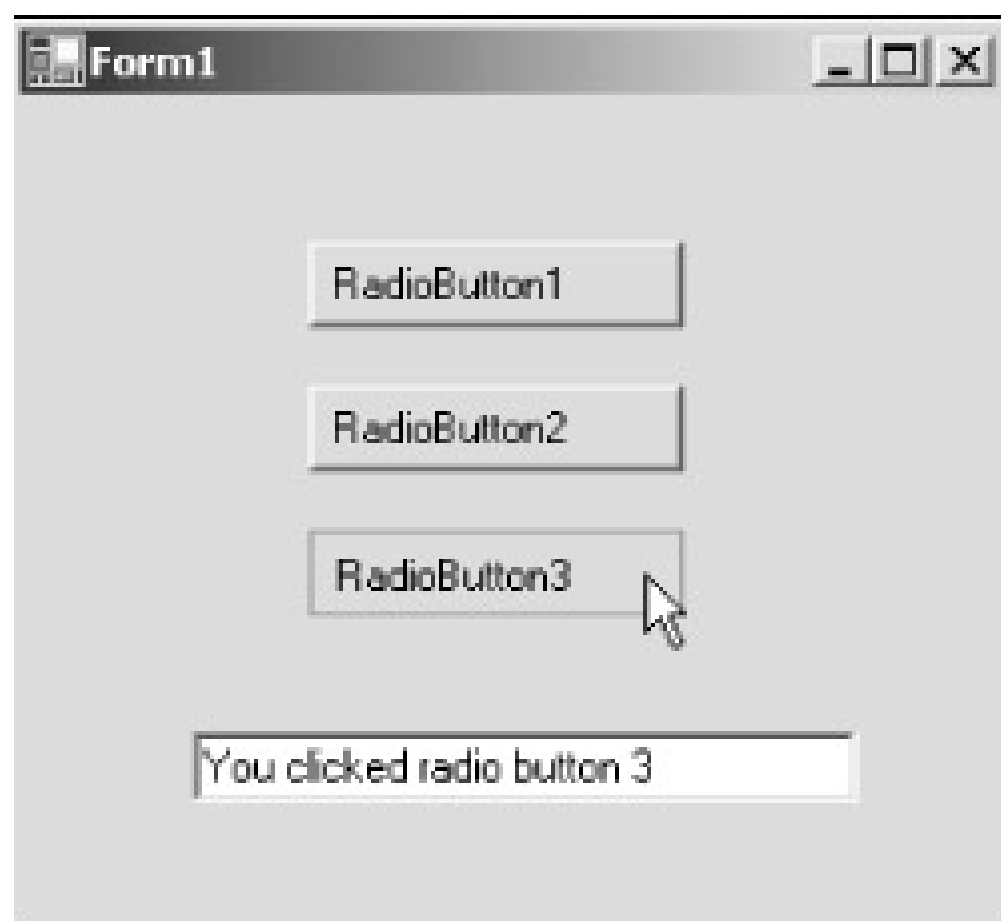
Here's an example

```
Private Sub Button1_Click(ByVal sender As System.Object, _  
    ByVal e As System.EventArgs) Handles Button1.Click  
    If RadioButton1.Checked Then  
        MsgBox ("The Radio Button is selected.")  
    Else  
        MsgBox ("The Radio Button is not selected.")  
    End If  
End Sub
```

## Creating Toggle Buttons

You can turn checkboxes or radio buttons into toggle buttons if you set their **Appearance** property to **Button** (the other option is **Normal**). Toggle buttons resemble standard buttons but act like the checkboxes or radio buttons they really are.

When you click a checkbox button, for example, it stays clicked until you click it again.



To change the appearance of radio buttons at run time run the following code

```
RadioButton1.Appearance = Appearance.Button
```

Using the *Panel* Class

panels are used to group controls together in a Windows form.

Noteworthy public properties of *Panel* objects are

|                           |                                                            |
|---------------------------|------------------------------------------------------------|
| <b>AutoScroll</b>         | Specifies if the panel will display scroll bars if needed. |
| <b>AutoScrollMargin</b>   | Gets/sets the size of the auto-scroll margin.              |
| <b>AutoScrollMinSize</b>  | Gets/sets the minimum size of the auto-scroll.             |
| <b>AutoScrollPosition</b> | Gets/sets the location of the auto-scroll position.        |
| <b>DockPadding</b>        | Gets the dock padding settings for all edges of the panel. |

Creating Panels

When you add radio buttons to a form, they are automatically coordinated so that only one radio button can be selected at a time. If the user selects a new radio button, all the other options buttons are automatically deselected. Some time have to select more than one button in such cases use panels



In the above diagram we can see that there are two panels containg radio buttons. One can select two radiobuttons at a time, one from each panel

Adding Controls to Panels in Code

You can add controls to a panel at run time just as you can add controls to a form. When you add controls to a form, you use the form's **Controls** collection; when you add controls to a panel, you use the panel's **Controls** collection. Here's an example,

```

Private Sub Button1_Click(ByVal sender As System.Object, _
    ByVal e As System.EventArgs) Handles Button1.Click
    Dim Panel1 As New Panel()
    Dim TextBox1 As New TextBox()
    Dim Label1 As New Label()

    Panel1.Location = New Point(60, 20)
    Panel1.Size = New Size(100, 150)
    Panel1.BorderStyle = System.Windows.Forms.BorderStyle.Fixed3D

    Label1.Location = New Point(16, 16)
    Label1.Text = "Enter text:"
    Label1.Size = New Size(60, 16)
    TextBox1.Location = New Point(16, 32)
    TextBox1.Text = ""
    TextBox1.Size = New Size(60, 20)

    Me.Controls.Add(Panel1)
    Panel1.Controls.Add(Label1)
    Panel1.Controls.Add(TextBox1)
End Sub

```

## Using the *GroupBox* Class

You can use group boxes to group controls together, much like panels. Unlike panels, however, group boxes can support captions, which you set with the **Text** property. However, group boxes cannot support scroll bars, which panels can.

## Creating Group Boxes

You can create group boxes at design time or run time. Group boxes do not have either a **BorderStyle** property, nor do they support scroll bars, as panels do.

## Adding Controls to Group Boxes in Code:

Use the group box's **Controls** collection's **Add** method to add controls to a group box.

Ex: `GroupBox1.Controls.Add(RadioButton1)`



